Sequelize + Express.js Cheat Sheet

1. Connect to DB

```
import { Sequelize } from 'sequelize';
import dotenv from 'dotenv'; dotenv.config();

const sequelize = new Sequelize(
  process.env.DB_NAME, process.env.DB_USER, process.env.DB_PASS,
  { host: process.env.DB_HOST, dialect:'mysql', port:3306,
logging:false }
);
export default sequelize;

// test:
try { await sequelize.authenticate(); }
catch(e){ console.error(e); }
finally { await sequelize.close(); }
```

2. Define model — `.define()`

```
import { DataTypes } from 'sequelize';
const Product = sequelize.define('Product', {
  name:  { type: DataTypes.STRING, allowNull:false },
  price: { type: DataTypes.FLOAT, defaultValue:0 },
  stock: { type: DataTypes.INTEGER, defaultValue:0 },
  genre: DataTypes.ENUM('fiction','sci-fi'),
});
export default Product;
```

Common types

STRING, TEXT, INTEGER, FLOAT, BOOLEAN, DATE, DATEONLY, ENUM(...)

Column opts

allowNull:false · defaultValue · unique:true · validate:{min,max}

3. Define model — class + `.init()`

```
import { Model, DataTypes } from 'sequelize';
class LibraryBook extends Model {
  getAgeInYears(){ return new Date().getFullYear()-
this.publishedYear; }
  isLongRead(){ return this.pages > 400; }
}
LibraryBook.init({
  title:{type:DataTypes.STRING,allowNull:false},
  pages: DataTypes.INTEGER,
}, { sequelize, modelName:'LibraryBook' });
```

Prototype instance methods

```
LibraryMember.prototype.getFullName = function(){
  return `${this.firstName} ${this.lastName}`;
};
member.getFullName();  // "Sokha Chan"
```

4. Sync models to DB

```
await sequelize.sync();                // create if missing
await sequelize.sync({ force:true }); // DROP + recreate (dev)
await sequelize.sync({ alter:true }); // adjust existing
```

force:true wipes data — dev only.

5. CRUD

```
// Create
await Product.create({name:'Pen',price:1.5});
await Product.bulkCreate([{name:'A'},{name:'B'}]);
// Read
await Product.findAll();
await Product.findByPk(1);
await Product.findOne({where:{name:'Pen'}});
await Product.count();
// Update (load+save)
const p = await Product.findByPk(1);
p.price = 12.99; await p.save();
// Update (bulk)
const [n] = await Product.update({stock:200},{where:{id:2}});
// Delete
await p.destroy();                     // instance
await Product.destroy({where:{price:{[Op.gt]:500}}}); // bulk
```

6. Query operators `Op`

```
import { Op } from 'sequelize';
```

Op	Meaning
eq/ne	= / !=
gt/gte	> / >=
lt/lte	< / <=
between	[min,max]
in/notIn	value in array
like/iLike	pattern match
and/or	combine conds

```
await Book.findAll({where:{price:{[Op.gt]:20}}});
await Book.findAll({where:{
  price:{[Op.between]:[10,40]},
  genre:{[Op.in]:['fiction','sci-fi']},
  stock:{[Op.gte]:1},
}});
```

7. Associations — 1:N and 1:1

```
// One-to-Many
Department.hasMany(Employee);
Employee.belongsTo(Department);
await Department.findAll({include:[Employee]});
dept.Employees              // array

// One-to-One
User.hasOne(Profile);
Profile.belongsTo(User);
const u = await User.findOne({where:{id},include:Profile});
u.Profile.bio

// custom FK
Student.hasMany(Attendance,{foreignKey:'studentId'});
Attendance.belongsTo(Student,{foreignKey:'studentId'});
```

8. Associations — Many-to-Many

```
Student.belongsToMany(Course,{through:Enrollment});
Course.belongsToMany(Student,{through:Enrollment});
await alice.addCourse(math,{through:{grade:'A'}});
const c = await alice.getCourses();
c[0].Enrollment.grade

// with alias (as) + extra data on join table
LibraryMember.belongsToMany(LibraryBook,
{through:Review,as:'ReviewBooks'});
LibraryBook.belongsToMany(LibraryMember,
{through:Review,as:'Reviewers'});
await member.addLibraryBook(book);
const favs = await member.getLibraryBooks();
await LibraryBook.findAll({include:
{model:LibraryMember,as:'Reviewers'}});
book.Reviewers[i].Review.rating   // join-table column
```

9. Join table with extra columns (Loan)

```
const Loan = sequelize.define('Loan', {
  borrowedAt: DataTypes.DATEONLY,
  returnedAt: { type:DataTypes.DATEONLY, allowNull:true },
});
await member.addBorrowedBooks(book, {
  through:{ borrowedAt:'2025-06-17', returnedAt:null }
});
const b = await member.getBorrowedBooks();
b[0].Loan.borrowedAt
```

10. Aggregating through a join table

```
const books = await LibraryBook.findAll({
  include:{model:LibraryMember,as:'Reviewers'}
});
for (const book of books){
  let sum=0;
  for (const r of book.Reviewers) sum += r.Review.rating;
  const avg = sum / book.Reviewers.length;
}
```

Same pattern reversed: loop member.ReviewedBooks, sum book.Review.rating, divide by .length.

11. Express server bootstrap

```
import express from 'express';
import cors from 'cors';
const app = express();
app.use(cors());
app.use(express.json());        // parse JSON body
app.use('/', routes);
await sequelize.sync({force:true});
app.listen(process.env.PORT||4000, ()=>console.log('API
running'));
```

12. Routes

```
import express from 'express';
const router = express.Router();

router.get('/students', async (req,res)=>{
  res.status(200).json(await Student.findAll());
});
router.post('/students', async (req,res)=>{
  const { name, email } = req.body;
  try {
    res.status(201).json(await Student.create({name,email}));
  } catch(err){ res.status(500).json({error:err.message}); }
});
export default router;
```

Reading requests

Source	Access
Body	req.body.field
Query ?a=1	req.query.a
URL /:id	req.params.id

13. Controllers (class-based)

```
class ProductController {
  static async createProduct(req,res){
    const {name,price,stock} = req.body;
    if(!name||!price||!stock)
      return res.status(400).json({error:'missing fields'});
    try{
      const p = await Product.create({name,price,stock});
      return
res.status(201).json({message:'created',product:p});
    }catch(err){ return
res.status(500).json({error:err.message}); }
  }
  static async getAllProducts(req,res){
    return res.status(200).json({products: await
Product.findAll()});
  }
}
export default ProductController;
// wiring:
router.get('/all', ProductController.getAllProducts);
router.post('/create', ProductController.createProduct);
```

14. Middleware

```
export const onlyPOST = (req,res,next)=>{
  if(req.method!=='POST')
    return res.status(405).json({error:'Only POST allowed'});
  next();
};
app.use(onlyPOST);                      // global
router.post('/x', onlyPOST, handler);   // per-route
```

15. HTTP status codes

Code	Meaning
200	OK (GET)
201	Created (POST)
400	Bad request
404	Not found
405	Method not allowed
500	Server error

```
try{ res.status(200).json(data); }
catch(err){ res.status(500).json({error:err.message}); }
```

16. Full endpoint example (borrow book)

```
export async function borrowBook(req,res){
  const {memberId,bookId} = req.query;
  try{
    const member = await LibraryMember.findByPk(memberId);
    const book = await LibraryBook.findByPk(bookId);
    await member.addBorrowedBooks(book,{
      through:{borrowedAt:new Date(),returnedAt:null}
    });
    res.status(201).json({message:'Book borrowed'});
  }catch(err){ res.status(500).json({error:err.message}); }
}
router.post('/loans', borrowBook); // POST /loans?
memberId=1&bookId=1
```

17. Eager loading (`include`)

```
await Attendance.findAll({ include:[Student, ClassRoom] });
await LibraryBook.findAll({
  include:{ model: LibraryMember, as:'Reviewers' }
});
// or via generated association methods (no include needed):
await member.getBorrowedBooks();
await member.getLibraryBooks();
```

18. .env + seed pattern

```
# .env
DB_NAME=mydb
DB_USER=root
DB_PASS=root
DB_HOST=localhost
PORT=4000
```

```
import dotenv from 'dotenv'; dotenv.config();
// idempotent seed
if (await Student.count() === 0) {
  await Student.create({name:'John',email:'j@x.com'});
}
```